

ФГАОУ ВО «НИУ ИТМО»  
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ  
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

## Лабораторная работа №2

### Системы линейных алгебраических уравнений

(по дисциплине «Вычислительная математика»)

**Выполнил:**

Лабин Макар Андреевич,  
ВЫЧМ 3.2, Р3231, 466449.

**Проверила:**

Перл Ольга Вячеславовна

**ИТМО**

г. Санкт-Петербург, Россия  
2026

# Содержание

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>1</b> | <b>Постановка задач</b>                | <b>1</b>  |
| <b>2</b> | <b>Выполнение работы</b>               | <b>2</b>  |
| 2.1      | Описание метода . . . . .              | 2         |
| 2.1.1    | Прямой ход . . . . .                   | 2         |
| 2.1.2    | Обратный ход . . . . .                 | 3         |
| 2.2      | Блок-схема численного метода . . . . . | 3         |
| 2.3      | Листинг численного метода . . . . .    | 5         |
| 2.3.1    | Метод Гаусса . . . . .                 | 5         |
| 2.3.2    | Программный интерфейс . . . . .        | 7         |
| 2.4      | Результаты работы программы . . . . .  | 7         |
| 2.4.1    | Внешний ввод данных . . . . .          | 7         |
| 2.4.2    | Случайные матрицы . . . . .            | 9         |
| <b>3</b> | <b>Заключение</b>                      | <b>11</b> |
| <b>4</b> | <b>Источники литературы</b>            | <b>12</b> |

# 1 Постановка задач

Дана система линейных алгебраических уравнений вида  $Ax = b$ , где  $A$  — матрица коэффициентов размерности  $n \times n$ , а  $b$  — вектор свободных членов.

В варианте №1 решение системы осуществляется *методом Гаусса*, а оценка корректности — через *расчёт вектора невязок*.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Разработка программного модуля для реализации алгоритма Гаусса.
2. Реализация проверки системы на совместность и определенность.
3. Написание алгоритма вычисления вектора невязок  $r$  для оценки погрешности полученного результата.
4. Вычисление вектора решений  $x$  с вектором невязок  $r$  для данной СЛАУ.

## 2 Выполнение работы

### 2.1 Описание метода

#### 2.1.1 Прямой ход

Рассмотрим общий вид СЛАУ размера  $n$  в матричном виде:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix} \iff A\vec{x} = \vec{f}$$

Метод Гаусса предлагает привести матрицу коэффициентов к *треугольному виду*. Для определённости будем рассматривать *верхнетреугольные матрицы*, то есть матрицы  $(u_{ij})$ , для которых верно:

$$i > j \implies u_{ij} = 0$$

Преобразование квадратной матрицы к верхнетреугольной возможно, если система имеет единственное решение. Покажем это конструктивным способом.

Пусть  $a_{11} \neq 0$ . Подействуем слева на равенство гомоморфизмом вида:

$$M_1 = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -\frac{a_{21}}{a_{11}} & 1 & 0 & \cdots & 0 \\ -\frac{a_{31}}{a_{11}} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\frac{a_{n1}}{a_{11}} & 0 & 0 & \cdots & 1 \end{pmatrix}$$

Это автоморфизм, поскольку  $\det(M_1) \neq 0$ . То есть шаг равносильен на  $\mathbb{R}^n$ .

Оператор  $M_1$  описывает прибавление к  $i$ -й строке первой строки, умноженной на коэффициент  $-a_{i1}/a_{11}$ . В результате получим систему вида:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(1)} & \cdots & a_{nn}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2^{(1)} \\ \vdots \\ f_n^{(1)} \end{pmatrix}$$

Пусть  $a_{11} = 0$ . Найдём среди  $a_{i1}$  ( $i > 1$ ) ненулевой элемент на  $j$ -ой строке. Подействуем слева на равенство гомоморфизмом вида:

$$P_{1j} = \begin{pmatrix} 0 & 0 & \cdots & 1 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 1 & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & \cdots & 1 \end{pmatrix},$$

где единица в первой строке стоит на  $j$ -м месте, а единица в  $j$ -й строке — на первом месте (*остальные диагональные элементы равны 1*).

Поскольку  $\det P_{1j} \neq 0$ , это преобразование также автоморфизм, что гарантирует сохранение множества решений системы.

Оператор  $P_{1j}$  описывает перестановку первой и  $j$ -ой строки местами. Так мы свели задачу к случаю  $\tilde{a}_{11} \neq 0$ .

Если весь первый столбец оказался нулевым, то базисный вектор  $e_1 = (1, 0, \dots, 0)^T$  принадлежит ядру оператора  $A$ . Это делает отображение неинъективным, а его матрицу — вырожденной. То есть решения у системы либо нет, либо их бесконечно много.

Далее рассмотрим сужение полученной композиции операторов на подпространство  $V_{n-1} \subset \mathbb{R}^n$ , которое является прямым дополнением к  $\text{span}\{e_1\}$ . Это соответствует переходу к работе со следующей системой:

$$\begin{pmatrix} a_{22}^{(1)} & a_{23}^{(1)} & \cdots & a_{2n}^{(1)} \\ a_{32}^{(1)} & a_{33}^{(1)} & \cdots & a_{3n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n2}^{(1)} & a_{n3}^{(1)} & \cdots & a_{nn}^{(1)} \end{pmatrix} \begin{pmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} f_2^{(1)} \\ f_3^{(1)} \\ \vdots \\ f_n^{(1)} \end{pmatrix}$$

Этот процесс повторяется итерационно для  $k = 2, 3, \dots, n-1$ . На каждом шаге мы выбираем ведущий элемент  $a_{kk}^{(k-1)}$ , при необходимости применяя автоморфизм перестановки строк, и конструируем соответствующий оператор  $M_k$ . В результате последовательного применения композиции автоморфизмов исходная матрица  $A$  преобразуется в верхнетреугольную матрицу  $U$ .

### 2.1.2 Обратный ход

Рассмотрим общий вид СЛАУ размера  $n$  в матричном виде с верхнетреугольной матрицей  $U$ , полученной после завершения прямого хода:

$$\begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \tilde{f}_1 \\ \tilde{f}_2 \\ \vdots \\ \tilde{f}_n \end{pmatrix}$$

Если все диагональные элементы  $u_{ii} \neq 0$ , система имеет единственное решение. Метод Гаусса предлагает выразить компоненты  $\vec{x}$  по следующей рекуррентной формуле:

$$x_i = \frac{1}{u_{ii}} \left( \tilde{f}_i - \sum_{j=i+1}^n u_{ij} x_j \right)$$

## 2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- $n$  — размерность системы; является целым числом.
- $A$  — расширенная матрица системы; является двумерным массивом вещественных чисел двойной точности.

На выходе метода ожидается вектор над вещественными числами двойной точности — решение системы, которое совмещено с таким же вектором невязки для данного решения.

В случае, если система не имеет единственного решения, метод должен выбросить соответствующую ошибку.

Далее приводятся блок-схемы, реализующие численный метод.

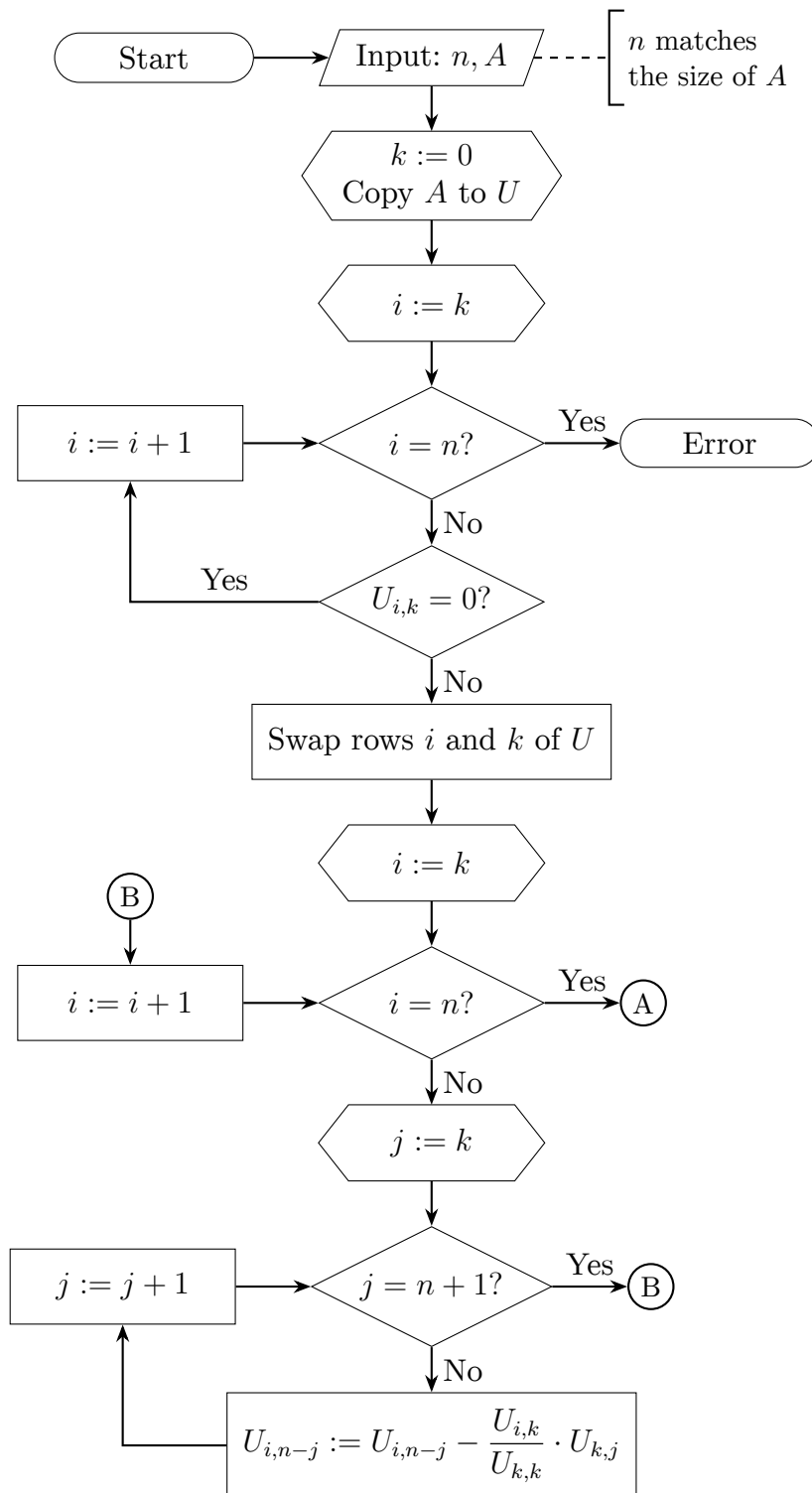
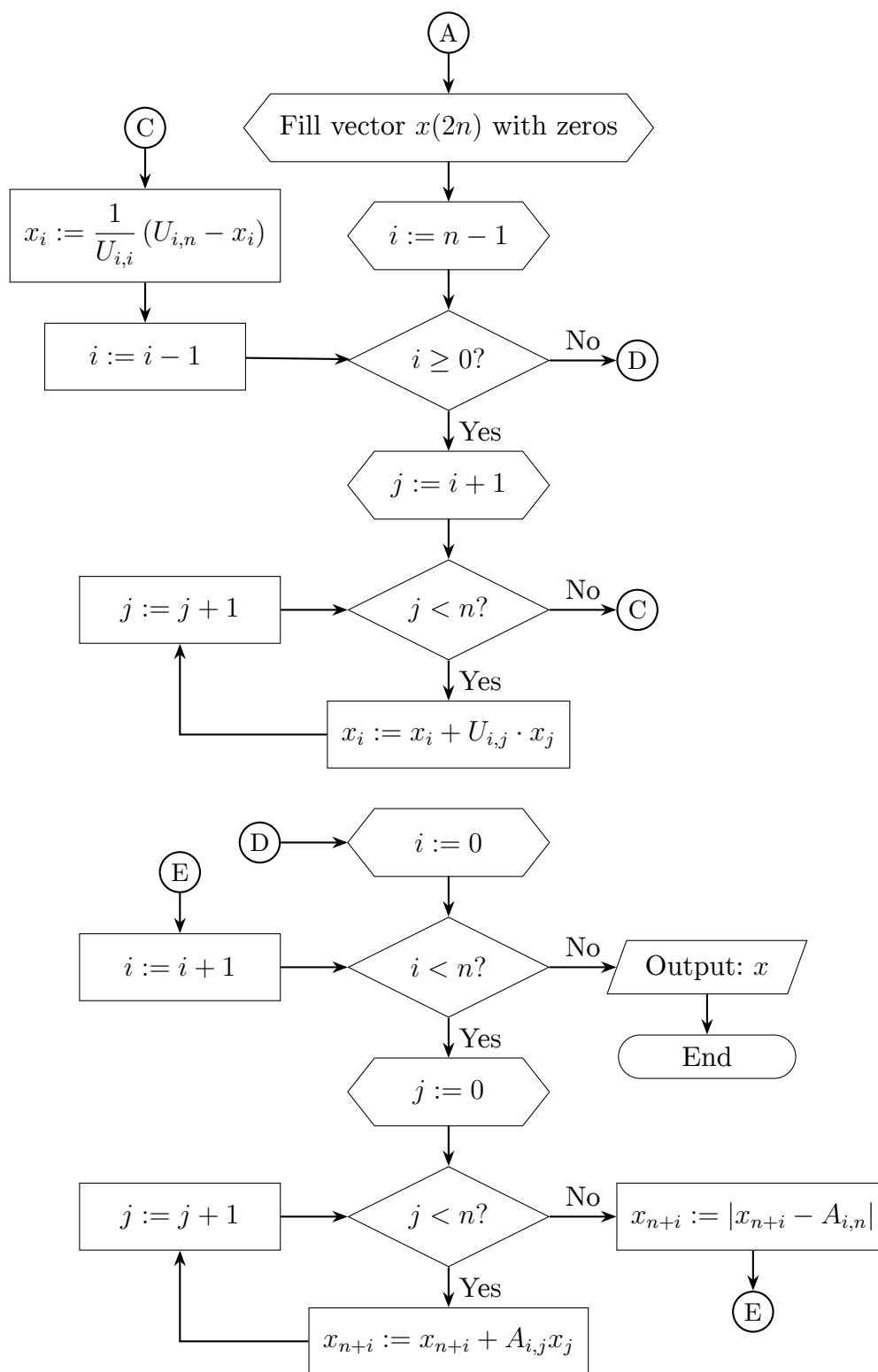


Схема 1 — Прямой ход метода Гаусса.



```

1 bool triangulateMatrix(const int n, vector<vector<double>>&
   matrix) {
2     bool isSingular = false;
3     for (size_t i = 0; i < n - 1; i++) {
4         size_t idx = n;
5         for (size_t j = i; j < n; j++) {
6             if (matrix[j][i] != 0) {
7                 idx = j;
8                 break;
9             }
10        }
11        if (idx == n) {
12            isSingular = true;
13            continue;
14        }
15        // apply P operator
16        if (idx != i) {
17            vector<double> temp_row = matrix[idx];
18            matrix[idx] = matrix[i];
19            matrix[i] = temp_row;
20        }
21        // apply M operator
22        for (size_t j = i + 1; j < n; j++) {
23            double factor = matrix[j][i] / matrix[i][i];
24            for (size_t k = i; k <= n; k++) {
25                matrix[j][k] -= factor * matrix[i][k];
26            }
27        }
28    }
29    if (isSingular) {
30        errorMessage = "The system has no single solution.";
31        isSolutionExists = !hasContradiction(n, matrix);
32        return false;
33    }
34    return true;
35 }

```

Листинг 1 — Прямой шаг метода Гаусса.

```

1 bool hasContradiction(const int n, vector<vector<double>>&
   matrix) {
2     vector<double> zeroes(n, 0);
3     for (size_t i = 0; i < n; i++) {
4         vector<double> curr_row = matrix[i];
5         curr_row.pop_back();
6         if (curr_row == zeroes && matrix[i][n] != 0) {
7             return true;
8         }
9     }
10    return false;
11 }

```

Листинг 2 — Проверка системы на совместность.



```

1 vector<double> solveByGauss(int n, vector<vector<double>> matrix
  ) {
2   vector<double> answer(2 * n, 0);
3   vector<vector<double>> matrix_triangle = matrix;
4   if (!triangulateMatrix(n, matrix_triangle)) {
5     return answer;
6   }
7   for (size_t i = 0; i < n; i++) {
8     const size_t idx = n - i - 1;
9     for (size_t j = idx + 1; j < n; j++) {
10      answer[idx] -= matrix_triangle[idx][j] * answer[j];
11    }
12    answer[idx] += matrix_triangle[idx][n];
13    answer[idx] /= matrix_triangle[idx][idx];
14  }
15  // calculate residuals
16  for (size_t i = 0; i < n; i++) {
17    for (size_t j = 0; j < n; j++) {
18      answer[n + i] += matrix[i][j] * answer[j];
19    }
20    answer[n + i] -= matrix[i][n];
21  }
22  outputTrace(matrix_triangle, answer);
23  return answer;
24 }

```

Листинг 3 — Обратный шаг метода Гаусса.

### 2.3.2 Программный интерфейс

...

## 2.4 Результаты работы программы

### 2.4.1 Внешний ввод данных

Для тестирования программы используем предложенные наборы данных:

1. Нулевые матрицы разных размерностей с ненулевым вектором  $f$ :

$$A_1 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}_{5 \times 5}, \quad f_1 = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$$

$$A_2 = \begin{pmatrix} 0 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 0 \end{pmatrix}_{10 \times 10}, \quad f_2 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 10 \end{pmatrix}$$

$$A_3 = \begin{pmatrix} 0 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 0 \end{pmatrix}_{20 \times 20}, \quad f_3 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 20 \end{pmatrix}$$

2. Матрицы со всеми элементами единицами разных размерностей,  $f$  также ненулевой:

$$A_4 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}_{5 \times 5}, \quad f_4 = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$$

$$A_5 = \begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}_{10 \times 10}, \quad f_5 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 10 \end{pmatrix}$$

$$A_6 = \begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}_{20 \times 20}, \quad f_6 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 20 \end{pmatrix}$$

3. Матрицы разных размерностей, у которых легко рассчитать определитель:

$$A_7 = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 10 \end{pmatrix}_{5 \times 5}, \quad f_7 = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 0 \\ 5 \end{pmatrix}$$

$$A_8 = \begin{pmatrix} 3 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 3 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 3 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 3 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 3 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 3 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 \end{pmatrix}_{10 \times 10}, \quad f_8 = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 10 \end{pmatrix}$$

$$A_9 = \begin{pmatrix} 5 & 1 & 0 & \cdots & 0 \\ 1 & 5 & 1 & \ddots & \vdots \\ 0 & 1 & 5 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & 5 \end{pmatrix}_{20 \times 20}, \quad f_9 = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

Проверим вывод для каждого набора данных:

| Матрица | Результат                                      |
|---------|------------------------------------------------|
| $A_1$   | Ошибка: система не имеет решений               |
| $A_2$   | Ошибка: система не имеет решений               |
| $A_3$   | Ошибка: система не имеет решений               |
| $A_4$   | Ошибка: система не имеет решений               |
| $A_5$   | Ошибка: система не имеет решений               |
| $A_6$   | Ошибка: система не имеет решений               |
| $A_7$   | Ошибка: система имеет бесконечно много решений |
| $A_8$   | Нулевая невязка                                |
| $A_9$   | Нулевая невязка                                |

Таблица 1: Результаты тестирования

### 2.4.2 Случайные матрицы

Для тестирования программы на случайных данных сгенерируем пять примеров разных размерностей. Укажем входные данные в их исходном формате.

1 ...

Листинг 4 — Расширенная матрица пяти переменных.

Результирующий вектор невязки:

...

1 ...

Листинг 5 — Расширенная матрица семи переменных.

Результирующий вектор невязки:

...

1 ...

Листинг 6 — Расширенная матрица десяти переменных.

Результирующий вектор невязки:

...

1 ...

Листинг 7 — Расширенная матрица четырнадцати переменных.

Результирующий вектор невязки:

...

1 ...

Листинг 8 — Расширенная матрица двадцати переменных.

Результирующий вектор невязки:

...

### 3 Заключение

В ходе выполнения лабораторной работы у меня возникали некоторые вопросы, на которые я смог найти ответ самостоятельно или в различных источниках. Поэтому приведу ответы на них здесь в качестве результата.

**Почему мы не стали «честно» решать СЛАУ?** Рассмотрим численный метод решения Крамера, сказать, что обусловленность высокая

Прийти к прямому методу решения СЛАУ

крч всё по Петрову и Лобанову

Достоинства: • Может быть применен в любом случае, если у системы есть решение и на главной диагонали нет 0 элементов. • Нет дополнительных подготовительных вычислений. • Результаты могут быть получены за конечное количество операций. • Низкая погрешность метода, если производить расчёты в последний момент (например, при вычислении вручную). Недостатки: • При вычислении на компьютере образуется значительная набегающая погрешность за счёт ограниченности разрядной сетки, особенно при работе с разреженными матрицами (коэффициенты близки к 0). • Ошибки накапливаются, т. к. расчёт производится последовательно. • Может занимать существенный объем памяти для больших матриц, так как всю матрицу нужно держать в памяти. • Алгоритмическая сложность метода  $O(n^3)$ , где  $n$  – число неизвестных.

Сравнить с другим методом из той же категории!!!

## 4 Источники литературы

- [1] Петров, И. Б. Лекции по вычислительной математике : учебное пособие / И. Б. Петров, А. И. Лобанов. — М. : Интернет-Университет Информационных Технологий ; БИНОМ. Лаборатория знаний, 2006. — 523 с. : ил., табл. — (Серия «Основы информационных технологий»).
- [2] Березин, И. С. Методы вычислений : в 2 т. / И. С. Березин, Н. П. Жидков. — 2-е изд., стереотип. — М. : Гос. изд-во физ.-мат. лит., 1962. — Т. 1. — 464 с.